SOFTWARE DETAILED DESIGN

Project IV

ABSTRACT

Minecraft Server Management Portal

Team Members

Abhinav,
Devki,
Liv,
Spencer

Table of Contents

System Level Design..... 2
- Description..... 2
- System Architecture Description..... 2

Client Design..... 3
- Description..... 3
- Client Software Architecture Description..... 3
- Graphical User Interface Design Description..... 3

CORE..... 4
- Class : APP..... 4
- Description..... 4
- Class : AppLayout..... 4
- Class : WebSocketClient..... 4

Features..... 5

User Interface..... 6

Server Design..... 7
- Description..... 7
- Server Software Architecture Description..... 7
- User Interface Design Description..... 7

Main..... 8
- Class: Main..... 8

Core..... 8
- Class: ServerState..... 8
- Class: ServerContext..... 8
- Class: Packet..... 9

App..... 9
- Class: AuthService..... 9
- Class: MinecraftServerService..... 9
- Class: StartupScriptService..... 10
- Class: WhitelistService..... 10
- Class: PropertiesService..... 10
- Class: BackupService..... 10
- Class: TelemetryService..... 11
- Class: TelemetryScheduler..... 11

Infra..... 11
- Class: AppConfig..... 11
- Class: WebSocketServer..... 12
- Class: PacketEncoder..... 12
- Class: PacketDecoder..... 12
- Class: PacketRouter..... 12
- Class: SystemMetricsReader..... 13
- Class: ChatLogReader..... 13

System Level Design

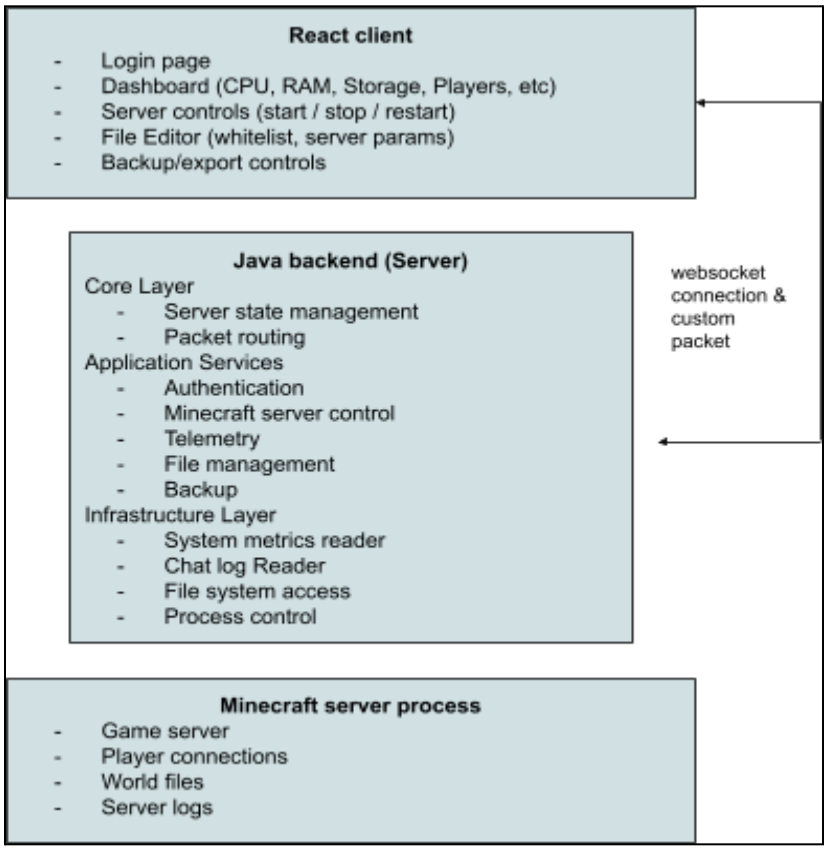
Description

The system is a Minecraft server management panel, designed to allow a user to monitor and control a Minecraft server remotely through a web interface. The system does this in two primary parts:

Server application (backend)
A Java application running on the same machine as the Minecraft server that is responsible for gathering telemetry information as well as modifying any files, and executing administrative commands. This includes, but is not limited to, changing the Java startup parameters, changing the whitelist, as well as starting, stopping, or restarting the server.

Client application (frontend)
A web application built in React that provides a GUI for users to interact with, as well as be able to view important information. This includes server status, resource usage, current players, the chat log, as well as any administrative actions like viewing/modifying configuration files or creating backups.

Communication between these two parts will be done with a custom packet protocol.



System Architecture Description

The system follows a client-server architecture. The React client acts as the user interface and sends commands to the backend server. The Java backend processes these commands, interacts with the Minecraft process and the operating system, and sends telemetry data back to the client.

A WebSocket connection is used instead of HTTP to allow real-time communication, ensuring continuous telemetry updates and immediate command responses.

Client Design

Description

The client application is designed as a web interface that allows the user to remotely manage and monitor a Minecraft server through the browser. The client is implemented using REACT and communicates with the backend using a WebSocket connection. The main objective of the client is to provide a clear and responsive interface for performing server management tasks such as monitoring resource usage and controlling server data. The client will interact with the backend server which will then perform the requested operations. The design emphasizes modular components, clear separation of responsibilities and real time updates from the server.

Client Software Architecture Description

The client software follows a modular component based architecture. The system is divided into 3 primary layers: Core, Features and UI. The core layer is responsible for application initialization, layout management and communication with the backend. This layer contains the root application component, layout structure and WebSocket communication manager. The features layer include main functional pages of the system including dashboard monitoring, server control, and backup management. Each feature is implemented as an independent module that interacts with the backend. The UI layer provides interface components and centralized application state. This layer is loosely coupled while maintaining clear data follow from the networking layer to the interface components.

Below is the simplified architectural structure of the client.

1. Core
    a. App
    b. AppLayout
    c. WebSocketClient
    d. ConnectionManager
2. Features
    a. LoginPage
    b. DashboardPage
    c. ServerControlPanel
    d. FileEditorPage
    e. BackupPage
3. UI
    a. TelemetryStore
    b. ConnectionStatusIndicator
    c. Reusable UI Components

Graphical User Interface Design Description

The graphical user interface is designed to provide a clear and efficient workflow for server administrators. The interface follows standard usability principles including Consistency, visibility

of system status and minimal cognitive load. The main menu uses a sidebar to let users switch between major options such as dashboard, server controls, and backup manager. The dashboard provides a real time overview of the system including CPU usage, player count and chat activity. Interaction elements are designed with clear affordances to indicate available actions.

CORE

Class : APP

Description

Root component that sets up the application and renders the layout.

Modules

- App bootstrap
- Route mounting
- Provider setup.

Class : AppLayout

Description

Main page layout containing navigation, header and content area.

Modules

- Sidebar area
- Header area
- Main content area

Class : WebSocketClient

Description

Handles the WebSocket connection to the server.

Modules

- connect()
- disconnect()
- sendPacket()
- onMessage()

Class : ConnectionManager

Description

Tracks whether the client is connected, disconnected or reconnecting.

Modules

- setStatus()
- getStatus()
- reconnect()

Features

Class : LoginPage

Description

Displays the login form and sends user credentials to server

Modules

- Username field
- Password field
- Submit button
- Error display

Class : DashboardPage

Description

Main monitoring page showing server state, resource usage, players and chat.

Modules

- Status section
- CPU/RAM/Storage section
- Players section
- Chat section

Class : ServerControlPanel

Description

UI for sending start, stop and restart commands to the server.

Modules

- Start button
- Stop button
- Restart button
- Current state display

Class : FileEditorPage

Description

UI for viewing and editing server files like whitelist or server.properties.

Modules

- File selector
- Text editor
- Save button
- validation/error message

Class : BackupPage

Description

UI for creating backups and exporting minecraft worlds.

Modules

- Create backup button
- Export world button
- Backup status display

User Interface

Class : TelemetryStore

Description

Stores the most recent telemetry data so the dashboard can update live.

Modules

- setTelemetry()
- getTelemetry()
- clearTelemetry()

Server Design

Description

The server is a Java application that runs on the same computer as the Minecraft server that is to be managed. The application will be able to start and stop the Minecraft server, change information in the server's directory such as its whitelist or settings, and query information about the server whilst it is running, such as the Minecraft JVM's RAM and CPU usage.

Server Software Architecture Description

The server will run as a classic Java application with a Core/App/Infra/Main structural design.

The application will have 4 operational states it will manage.
The first state is STARTUP, which is when the user will be able to tell it where the Minecraft server directory is located, so that the application can find the necessary files inside of it.
The second state, STOPPED, is when Minecraft is not running. This will allow the user to change the whitelist file of the server, the server.properties file, the startup parameters (such as maximum RAM usage), and start a large file transfer which will zip the Minecraft world/ directory and send it to the client.
The third state, RUNNING, is when a Minecraft instance is running. This will allow the user to get recurring telemetry data sent to them, such as CPU usage, RAM usage, and Minecraft specific details like active player count and chat history.
The fourth state is BLOCKED, which is when the Minecraft instance is starting up or shutting down. This state is caused when the RUNNING state gives the "Stop server" command, or when the STOPPED state gives the "Start server" command. No operations will be able to take place during this time.

The server will expose an API that the client can connect to through websockets. After the client goes through a login authorization process, it will be sent the data from the server, including the state, and a custom packet of information following depending on the state of the server.

The server and client will share a custom packet design to send telemetry data and other files between each other.

User Interface Design Description

The user interface for the server is going to be a Command Line Interface with a simple menu system.

Its usage will match that of the client, where you can use the command line to also interface with the Minecraft server to perform start/stop operations, RAM/CPU/player count/chat queries, and change file info, but you will also be able to change the Minecraft server directory, and view a detailed log of all of the packets sent between the server and the client.

The startup page will simply be a line asking the user to input the absolute path of the minecraft server directory.

The menu will have the following structure

Current state: RUNNING

1. Change minecraft directory
2. View packet log
3. View server status
4. Modify server files
5. Stop server

Main

Class: Main

Description

Entry point of the application

Modules or Variables

main()

Core

Class: ServerState

Description

Enum that holds the minecraft server state, such as startup, stopped, running, and blocked.

Modules or Variables

STATUS enum

Class: ServerContext

Description

Class that holds the runtime state for the application, such as server state, minecraft path defined in setup, and the process ID.

Modules or Variables

get and setState()
get and setMinecraftPath()
getCurrentProcessId()

Class: Packet

Description

Model of our custom packet that'll be shared between client and server.

Modules or Variables

Header
Body
CRC
Source
Destination
Command

App

Class: AuthService

Description

Handles login validation, client connection, and client permissions.

Modules or Variables

login()
validateCredentials()
isAuthorized()

Class: MinecraftServerService

Description

Main Minecraft service for starting/stopping/restarting the Minecraft server

Modules or Variables

startServer()
stopServer()

restartServer()
isRunning()

Class: StartupScriptService

Description

Handles the startup parameters for the minecraft server (max ram, GC rules, etc)

Modules or Variables

readStartupScript()
writeStartupScript()
updateBootParameters()

Class: WhitelistService

Description

Handles reading and changing the Minecraft server whitelist (allowed players).

Modules or Variables

readWhitelist()
writeWhitelist()
addPlayer()
removePlayer()

Class: PropertiesService

Description

Handles reading and changing the Minecraft server.properties file (server settings)

Modules or Variables

readProperties()
writeProperties()
updateProperties()

Class: BackupService

Description

Handles creation and sending of the Minecraft world folder backup (zipping and sending to client)

Modules or Variables

createBackup()
listBackups()
exportWorld()

Class: TelemetryService

Description

Handles collection of current system and minecraft server information and packages it to send to the client.

Modules or Variables

buildSnapshot()
getCurrentTelemetry()
collectSystemData()
collectMinecraftData()

Class: TelemetryScheduler

Description

Handles collecting aforementioned telemetry on a timed cycle.

Modules or Variables

startPolling()
stopPolling()
broadcastTelemetry()

Infra

Class: AppConfig

Description

Loads and stores app settings (ports, file paths, polling intervals)

Modules or Variables

loadConfig()
saveConfig()
getPort()
getDataPaths()

Class: WebSocketServer

Description

Manages client connections and transports packets between client and server app.

Modules or Variables

start()
stop()
onConnect()
onDisconnect()
onMessage()
sendMessage()

Class: PacketEncoder

Description

Converts the packet object into the outgoing string format.

Modules or Variables

encode()
buildHeader()
appendCRC()

Class: PacketDecoder

Description

Converts (parses) the incoming string format into a packet object

Modules or Variables

decode()
validatePacket()
extractHeader()
extractBody()

Class: PacketRouter

Description

Inspects incoming commands and routes them to the correct services.

Modules or Variables

route()
handleAuth()
handleServerControl()
handleFileRequest()
handleTelemetryRequest()

Class: SystemMetricsReader

Description

Reads machine specific telemetry (CPU, RAM, Storage usage)

Modules or Variables

readCpuUsage()
readRamUsage()
readDiskUsage()

Class: ChatLogReader

Description

Reads Minecraft chat lines by tailing the latest.log file to send to the client.

Modules or Variables

readNewLines()
parseChatMessages()
trackFileOffset()